Separation Logic II

Inductive predicates

Cristina David

Lecture 11

COMSM0067 — Advanced Topics in Programming Languages
University of Bristol

Where we are

Last time: $*$, and the frame rule. We can talk about one cell, and frame off the rest.

Where we are

Last time: $*$, and the frame rule. We can talk about one cell, and frame off the rest.

But $x \mapsto v$ describes a fixed, finite heap. A linked list has unboundedly many cells:



We cannot write

$$x \mapsto _ * _ \mapsto _ * _ \mapsto _ * \cdots$$

Where we are

Last time: $*$, and the frame rule. We can talk about one cell, and frame off the rest.

But $x \mapsto v$ describes a fixed, finite heap. A linked list has unboundedly many cells:



We cannot write

$$x \mapsto _ * _ \mapsto _ * _ \mapsto _ * \cdots$$

We need recursive heap assertions.

Inductive predicates

Linked lists, inductively

A cell $x \mapsto v, z$ holds a value v and a next-pointer z:



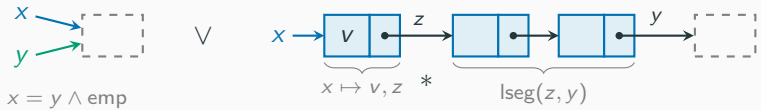
Linked lists, inductively

A cell $x \mapsto v, z$ holds a value v and a next-pointer z:



List segment from x to y

$$\mathsf{lseg}(x, y) \equiv (x = y \wedge \mathsf{emp}) \vee (x \neq y \wedge \exists v, z.\ x \mapsto v, z * \mathsf{lseg}(z, y))$$



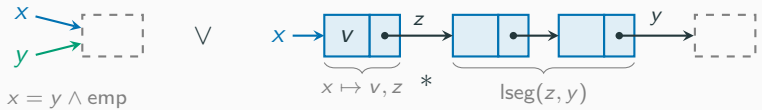
Linked lists, inductively

A cell $x \mapsto v, z$ holds a value v and a next-pointer z:



List segment from x to y

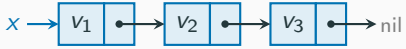
$$\mathsf{lseg}(x, y) \equiv (x = y \wedge \mathsf{emp}) \vee (x \neq y \wedge \exists v, z.\ x \mapsto v, z * \mathsf{lseg}(z, y))$$



Note the $*$ in the recursive case: the head cell is disjoint from the rest of the list. Separation is baked into the datatype — that is what makes it a *well-formed* list and not a tangle of aliases.

Question: a full list

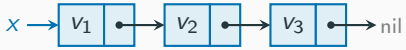
A full list is null-terminated: follow the next-pointers from x and you reach nil.



How would you define list(x) using lseg?

Question: a full list

A full list is null-terminated: follow the next-pointers from x and you reach nil.



How would you define list(x) using lseg?

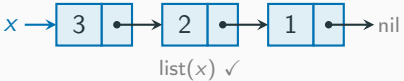
$$\mathsf{list}(x) \;\equiv\; \mathsf{lseg}(x, \mathsf{nil})$$

A full list is a segment that ends at nil. Including the empty list: $x = \mathsf{nil}$, no cells.

list forgets the contents

Both heaps satisfy list(x):

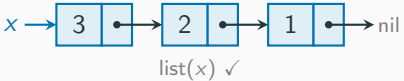




list forgets the contents

Both heaps satisfy list(x):





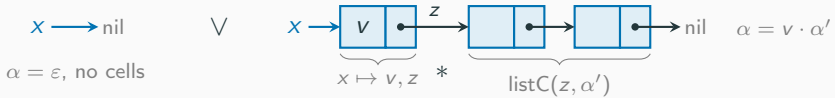
Enough for memory safety. Not for reversal: we must say *which* list comes out.

Tracking the *contents*

Index the predicate by the sequence of values α it holds:

List with contents

$$\mathsf{listC}(x, \alpha) \;\equiv\; \big(x = \mathsf{nil} \wedge \alpha = \varepsilon \wedge \mathsf{emp}\big) \vee \big(\exists v, z, \alpha'.\, \alpha = v{\cdot}\alpha' \wedge x \mapsto v, z * \mathsf{listC}(z, \alpha')\big)$$



ε: the empty sequence. $v \cdot \alpha'$: v in front of α'.

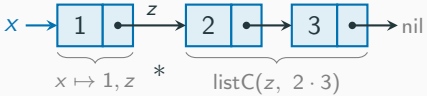
listC on an example



This heap satisfies listC(x, $1 \cdot 2 \cdot 3$).

What do we get if we unfold it once?

listC on an example

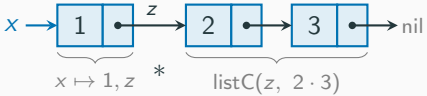


This heap satisfies $\mathsf{listC}(x,\ 1 \cdot 2 \cdot 3)$.

What do we get if we unfold it once?

$$\mathsf{listC}(x,\ 1 \cdot 2 \cdot 3) \ =\ \exists z.\ x \mapsto 1, z \ *\ \mathsf{listC}(z,\ 2 \cdot 3)$$

listC on an example



This heap satisfies $\mathsf{listC}(x,\ 1 \cdot 2 \cdot 3)$.

What do we get if we unfold it once?

$$\mathsf{listC}(x,\ 1 \cdot 2 \cdot 3)\ =\ \exists z.\ x \mapsto 1, z\ *\ \mathsf{listC}(z,\ 2 \cdot 3)$$

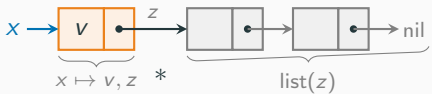
Forget the contents and you get list back:

$$\mathsf{list}(x)\ \equiv\ \exists \alpha.\ \mathsf{listC}(x, \alpha)$$

Unfolding is the engine in proofs

A predicate is used by unfolding its definition — one step at a time. If we know $x \neq \mathsf{nil}$, then $\mathsf{list}(x)$ unfolds to

$$\exists v, z.\ x \mapsto v, z \ * \ \mathsf{list}(z)$$

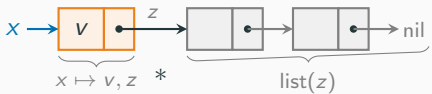


One concrete head cell, exposed; the rest is a shorter list, still folded.

Unfolding is the engine in proofs

A predicate is used by unfolding its definition — one step at a time. If we know $x \neq \text{nil}$, then $\text{list}(x)$ unfolds to

$$\exists v, z.\ x \mapsto v, z \ * \ \text{list}(z)$$

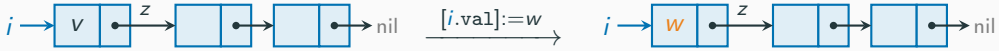


One concrete head cell, exposed; the rest is a shorter list, still folded.

Unfold at the front, operate on the exposed cell, fold back up.

Every list proof is this rhythm. The $*$ ensures the exposed cell and the tail never alias — so we can mutate the head and frame the tail.

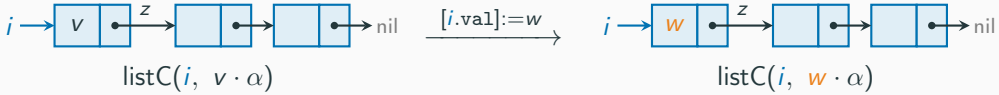
Example: overwrite the head



$\mathsf{listC}(i,\ v \cdot \alpha)$ $\mathsf{listC}(i,\ w \cdot \alpha)$

$$\{\mathsf{listC}(i,\ v \cdot \alpha)\}\ \ [i.\mathtt{val}] := w\ \ \{\mathsf{listC}(i,\ w \cdot \alpha)\}$$

Example: overwrite the head

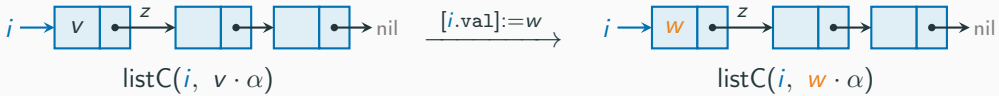


$$\{\mathsf{listC}(i,\ v \cdot \alpha)\}\ \ [i.\mathtt{val}] := w\ \ \{\mathsf{listC}(i,\ w \cdot \alpha)\}$$

No loop, so no invariant: unfold, operate, fold.

$$\{\mathsf{listC}(i,\ v \cdot \alpha)\}$$

Example: overwrite the head



$\mathsf{listC}(i,\ v \cdot \alpha)$ $\mathsf{listC}(i,\ w \cdot \alpha)$

$$\{\mathsf{listC}(i,\ v \cdot \alpha)\}\ \ [i.\mathtt{val}] := w\ \ \{\mathsf{listC}(i,\ w \cdot \alpha)\}$$

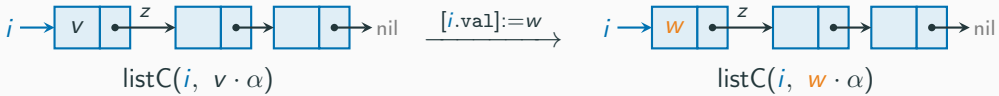
No loop, so no invariant: unfold, operate, fold.

$\{\mathsf{listC}(i,\ v \cdot \alpha)\}$

$\{i \mapsto v, z * \mathsf{listC}(z, \alpha)\}$ unfold (sequence is non-empty)

$[i.\mathtt{val}] := w$ mutation, frame $\mathsf{listC}(z, \alpha)$

Example: overwrite the head



$$\{\mathsf{listC}(i,\ v \cdot \alpha)\}\ \ [i.\mathtt{val}] := w\ \ \{\mathsf{listC}(i,\ w \cdot \alpha)\}$$

No loop, so no invariant: unfold, operate, fold.

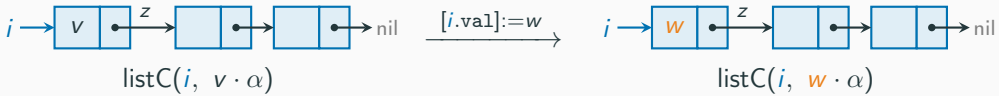
$\{\mathsf{listC}(i,\ v \cdot \alpha)\}$

$\{i \mapsto v, z * \mathsf{listC}(z, \alpha)\}$ unfold (sequence is non-empty)

$[i.\mathtt{val}] := w$ mutation, frame $\mathsf{listC}(z, \alpha)$

$\{i \mapsto w, z * \mathsf{listC}(z, \alpha)\}$

Example: overwrite the head



$$\{\mathsf{listC}(i,\ v \cdot \alpha)\}\ \ [i.\mathtt{val}] := w\ \ \{\mathsf{listC}(i,\ w \cdot \alpha)\}$$

No loop, so no invariant: unfold, operate, fold.

$\{\mathsf{listC}(i,\ v \cdot \alpha)\}$

$\{i \mapsto v, z * \mathsf{listC}(z, \alpha)\}$ unfold (sequence is non-empty)

$[i.\mathtt{val}] := w$ mutation, frame $\mathsf{listC}(z, \alpha)$

$\{i \mapsto w, z * \mathsf{listC}(z, \alpha)\}$

$\{\mathsf{listC}(i,\ w \cdot \alpha)\}$ fold

In-place list reversal

In-place list reversal



listC(i, $1 \cdot 2 \cdot 3$) listC(j, $3 \cdot 2 \cdot 1$)

$\{\text{listC}(i, \alpha_0)\}$

```
j := nil;
while i ≠ nil do {
  k := [i.next];      // remember the rest
  [i.next] := j;      // flip this link
  j := i;             // grow reversed part
  i := k              // advance
}
```

$\{\text{listC}(j,\ \text{rev}(\alpha_0))\}$

- Same cells, every link flipped. No allocation.

In-place list reversal



listC(i, $1 \cdot 2 \cdot 3$) listC(j, $3 \cdot 2 \cdot 1$)

$\{\text{listC}(i, \alpha_0)\}$

```
j := nil;
while i ≠ nil do {
  k := [i.next];      // remember the rest
  [i.next] := j;      // flip this link
  j := i;             // grow reversed part
  i := k              // advance
}
```

$\{\text{listC}(j,\ \text{rev}(\alpha_0))\}$

- Same cells, every link flipped. No allocation.
- $\mapsto$, $*$ are exact: no leaks, no stray cells.

In-place list reversal



listC(i, $1 \cdot 2 \cdot 3$) listC(j, $3 \cdot 2 \cdot 1$)

$\{\text{listC}(i, \alpha_0)\}$

```
j := nil;
while i ≠ nil do {
  k := [i.next];      // remember the rest
  [i.next] := j;      // flip this link
  j := i;             // grow reversed part
  i := k              // advance
}
```

$\{\text{listC}(j,\ \text{rev}(\alpha_0))\}$

- Same cells, every link flipped. No allocation.
- $\mapsto$, $*$ are exact: no leaks, no stray cells.
- i walks the remaining list; j grows the reversed part.

In-place list reversal



listC(i, $1 \cdot 2 \cdot 3$) listC(j, $3 \cdot 2 \cdot 1$)

$\{\text{listC}(i, \alpha_0)\}$

```
j := nil;
while i ≠ nil do {
  k := [i.next];      // remember the rest
  [i.next] := j;      // flip this link
  j := i;             // grow reversed part
  i := k              // advance
}
```

$\{\text{listC}(j,\ \text{rev}(\alpha_0))\}$

- Same cells, every link flipped. No allocation.
- $\mapsto$, $*$ are exact: no leaks, no stray cells.
- i walks the remaining list; j grows the reversed part.

What is the loop invariant?

The invariant

Let α_0 be the original contents. Here is the heap after two laps on $\alpha_0 = 1 \cdot 2 \cdot 3 \cdot 4$:



The invariant

Let α_0 be the original contents. Here is the heap after two laps on $\alpha_0 = 1 \cdot 2 \cdot 3 \cdot 4$:



Loop invariant

$$\exists \alpha, \beta. \quad \mathsf{listC}(i, \alpha) \ * \ \mathsf{listC}(j, \beta) \quad \wedge \quad \alpha_0 = \mathrm{rev}(\beta) \cdot \alpha$$

The invariant

Let α_0 be the original contents. Here is the heap after two laps on $\alpha_0 = 1 \cdot 2 \cdot 3 \cdot 4$:



Loop invariant

$$\exists \alpha, \beta. \quad \mathsf{listC}(i, \alpha) \ * \ \mathsf{listC}(j, \beta) \quad \wedge \quad \alpha_0 = \mathrm{rev}(\beta) \cdot \alpha$$

- Two disjoint lists: $*$ says i and j share no cells, so the surgery on one cannot corrupt the other.
- α is what's left to do; β is what's done, reversed. Here: $\mathrm{rev}(2 \cdot 1) \cdot 3 \cdot 4 = \alpha_0$ ✓

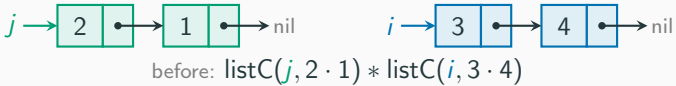
One lap, pictured



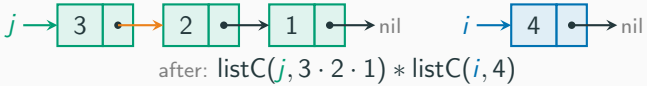
before: $\mathsf{listC}(j, 2 \cdot 1) * \mathsf{listC}(i, 3 \cdot 4)$

```
k := [i.next]; [i.next] := j; j := i; i := k
```

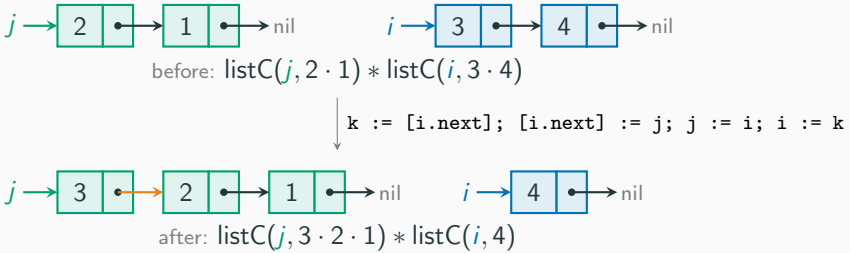
One lap, pictured




```
k := [i.next]; [i.next] := j; j := i; i := k
```



One lap, pictured



The head cell of i's list moves to the front of j's list. One link flips; every other cell is untouched.

Invariant again: $\mathrm{rev}(3 \cdot 2 \cdot 1) \cdot 4 = 1 \cdot 2 \cdot 3 \cdot 4 = \alpha_0$ ✓

The proof, whole program

Write I for the invariant $\exists \alpha, \beta.\ \mathsf{listC}(i, \alpha) * \mathsf{listC}(j, \beta) \wedge \alpha_0 = \mathrm{rev}(\beta) \cdot \alpha$.

$\{\mathsf{listC}(i, \alpha_0)\}$
  $j := \mathsf{nil};$
$\{\mathsf{listC}(i, \alpha_0) \wedge j = \mathsf{nil}\}$
$\{I\}$ VC1
  while $i \neq \mathsf{nil}$ do
    $\{I \wedge i \neq \mathsf{nil}\}$ *body* $\{I\}$ VC2
$\{I \wedge i = \mathsf{nil}\}$
$\{\mathsf{listC}(j,\ \mathrm{rev}(\alpha_0))\}$ VC3

The proof, whole program

Write I for the invariant $\exists \alpha, \beta.\ \mathsf{listC}(i, \alpha) * \mathsf{listC}(j, \beta) \wedge \alpha_0 = \mathrm{rev}(\beta) \cdot \alpha$.

$\{\mathsf{listC}(i, \alpha_0)\}$
  $j := \mathsf{nil};$
$\{\mathsf{listC}(i, \alpha_0) \wedge j = \mathsf{nil}\}$
$\{I\}$ VC1
  while $i \neq \mathsf{nil}$ do
    $\{I \wedge i \neq \mathsf{nil}\}$ *body* $\{I\}$ VC2
$\{I \wedge i = \mathsf{nil}\}$
$\{\mathsf{listC}(j,\ \mathrm{rev}(\alpha_0))\}$ VC3

Verification conditions

VC1 init: $\mathsf{listC}(i, \alpha_0) \wedge j = \mathsf{nil} \Rightarrow I$

VC2 preserve: $\{I \wedge i \neq \mathsf{nil}\}$ *body* $\{I\}$

VC3 exit: $I \wedge i = \mathsf{nil} \Rightarrow \mathsf{listC}(j,\ \mathrm{rev}(\alpha_0))$

Proving VC1 and VC3

$I \equiv \exists \alpha, \beta.\ \mathsf{listC}(i, \alpha) * \mathsf{listC}(j, \beta) \wedge \alpha_0 = \mathrm{rev}(\beta) \cdot \alpha$

VC1: $\mathsf{listC}(i, \alpha_0) \wedge j = \mathsf{nil} \ \Rightarrow\ I$

$\mathsf{listC}(i, \alpha_0) \wedge j = \mathsf{nil}$

$\Rightarrow\ \mathsf{listC}(i, \alpha_0) * \mathsf{listC}(j, \varepsilon)$ fold: $\mathsf{listC}(\mathsf{nil}, \varepsilon)$ is emp

$\Rightarrow\ I$ take $\alpha = \alpha_0,\ \beta = \varepsilon$: $\mathrm{rev}(\varepsilon) \cdot \alpha_0 = \alpha_0$

Proving VC1 and VC3

$I \equiv \exists \alpha, \beta.\ \mathsf{listC}(i, \alpha) * \mathsf{listC}(j, \beta) \wedge \alpha_0 = \mathrm{rev}(\beta) \cdot \alpha$

VC1: $\mathsf{listC}(i, \alpha_0) \wedge j = \mathsf{nil} \ \Rightarrow\ I$

$\mathsf{listC}(i, \alpha_0) \wedge j = \mathsf{nil}$

$\Rightarrow\ \mathsf{listC}(i, \alpha_0) * \mathsf{listC}(j, \varepsilon)$ fold: $\mathsf{listC}(\mathsf{nil}, \varepsilon)$ is emp

$\Rightarrow\ I$ take $\alpha = \alpha_0,\ \beta = \varepsilon$: $\mathrm{rev}(\varepsilon) \cdot \alpha_0 = \alpha_0$

VC3: $I \wedge i = \mathsf{nil} \ \Rightarrow\ \mathsf{listC}(j,\ \mathrm{rev}(\alpha_0))$

$\mathsf{listC}(\mathsf{nil}, \alpha) * \mathsf{listC}(j, \beta) \wedge \alpha_0 = \mathrm{rev}(\beta) \cdot \alpha$

$\Rightarrow\ \mathsf{listC}(j, \beta) \wedge \alpha_0 = \mathrm{rev}(\beta)$ unfold $\mathsf{listC}(\mathsf{nil}, \alpha)$: $\alpha = \varepsilon,\ \mathsf{emp}$

$\Rightarrow\ \mathsf{listC}(j,\ \mathrm{rev}(\alpha_0))$ $\beta = \mathrm{rev}(\mathrm{rev}(\beta)) = \mathrm{rev}(\alpha_0)$

Unfolding $\mathsf{listC}(\mathsf{nil}, \alpha)$ leaves only the empty case: nil is never allocated.

Proving VC2: one lap

$\{\mathsf{listC}(i, \alpha) * \mathsf{listC}(j, \beta) \wedge i \neq \mathsf{nil}\}$ from $I \wedge i \neq \mathsf{nil}$

$\{i \mapsto v, z * \mathsf{listC}(z, \alpha') * \mathsf{listC}(j, \beta)\}$ unfold, $\alpha = v \cdot \alpha'$

$k := [i.\mathtt{next}];$ lookup, frame $\mathsf{listC}(z, \alpha') * \mathsf{listC}(j, \beta)$

Proving VC2: one lap

$\{\mathsf{listC}(i, \alpha) * \mathsf{listC}(j, \beta) \wedge i \neq \mathsf{nil}\}$ from $I \wedge i \neq \mathsf{nil}$

$\{i \mapsto v, z * \mathsf{listC}(z, \alpha') * \mathsf{listC}(j, \beta)\}$ unfold, $\alpha = v \cdot \alpha'$

$k := [i.\mathtt{next}];$ lookup, frame $\mathsf{listC}(z, \alpha') * \mathsf{listC}(j, \beta)$

$\{i \mapsto v, k * \mathsf{listC}(k, \alpha') * \mathsf{listC}(j, \beta)\}$ using $k = z$

$[i.\mathtt{next}] := j;$ mutation, frame $\mathsf{listC}(k, \alpha') * \mathsf{listC}(j, \beta)$

Proving VC2: one lap

$\{\mathsf{listC}(i,\alpha) * \mathsf{listC}(j,\beta) \wedge i \neq \mathsf{nil}\}$	from $I \wedge i \neq \mathsf{nil}$
$\{i \mapsto v, z * \mathsf{listC}(z,\alpha') * \mathsf{listC}(j,\beta)\}$	unfold, $\alpha = v \cdot \alpha'$
$k := [i.\mathtt{next}];$	lookup, frame $\mathsf{listC}(z,\alpha') * \mathsf{listC}(j,\beta)$
$\{i \mapsto v, k * \mathsf{listC}(k,\alpha') * \mathsf{listC}(j,\beta)\}$	using $k = z$
$[i.\mathtt{next}] := j;$	mutation, frame $\mathsf{listC}(k,\alpha') * \mathsf{listC}(j,\beta)$
$\{i \mapsto v, j * \mathsf{listC}(k,\alpha') * \mathsf{listC}(j,\beta)\}$	
$\{\mathsf{listC}(k,\alpha') * \mathsf{listC}(i,\ v \cdot \beta)\}$	fold: $i \mapsto v, j * \mathsf{listC}(j,\beta)$
$j := i; \quad i := k$	assignment, twice

Proving VC2: one lap

$\{\mathsf{listC}(i,\alpha) * \mathsf{listC}(j,\beta) \wedge i \neq \mathsf{nil}\}$	from $I \wedge i \neq \mathsf{nil}$
$\{i \mapsto v, z * \mathsf{listC}(z,\alpha') * \mathsf{listC}(j,\beta)\}$	unfold, $\alpha = v \cdot \alpha'$
$k := [i.\mathtt{next}];$	lookup, frame $\mathsf{listC}(z,\alpha') * \mathsf{listC}(j,\beta)$
$\{i \mapsto v, k * \mathsf{listC}(k,\alpha') * \mathsf{listC}(j,\beta)\}$	using $k = z$
$[i.\mathtt{next}] := j;$	mutation, frame $\mathsf{listC}(k,\alpha') * \mathsf{listC}(j,\beta)$
$\{i \mapsto v, j * \mathsf{listC}(k,\alpha') * \mathsf{listC}(j,\beta)\}$	
$\{\mathsf{listC}(k,\alpha') * \mathsf{listC}(i,\ v \cdot \beta)\}$	fold: $i \mapsto v, j * \mathsf{listC}(j,\beta)$
$j := i; \quad i := k$	assignment, twice
$\{\mathsf{listC}(i,\alpha') * \mathsf{listC}(j,\ v \cdot \beta)\}$	

Proving VC2: one lap

$\{\mathsf{listC}(i,\alpha) * \mathsf{listC}(j,\beta) \wedge i \neq \mathsf{nil}\}$ — from $I \wedge i \neq \mathsf{nil}$

$\{i \mapsto v, z * \mathsf{listC}(z,\alpha') * \mathsf{listC}(j,\beta)\}$ — unfold, $\alpha = v \cdot \alpha'$

$k := [i.\mathtt{next}];$ — lookup, frame $\mathsf{listC}(z,\alpha') * \mathsf{listC}(j,\beta)$

$\{i \mapsto v, k * \mathsf{listC}(k,\alpha') * \mathsf{listC}(j,\beta)\}$ — using $k = z$

$[i.\mathtt{next}] := j;$ — mutation, frame $\mathsf{listC}(k,\alpha') * \mathsf{listC}(j,\beta)$

$\{i \mapsto v, j * \mathsf{listC}(k,\alpha') * \mathsf{listC}(j,\beta)\}$

$\{\mathsf{listC}(k,\alpha') * \mathsf{listC}(i,\ v \cdot \beta)\}$ — fold: $i \mapsto v, j * \mathsf{listC}(j,\beta)$

$j := i; \quad i := k$ — assignment, twice

$\{\mathsf{listC}(i,\alpha') * \mathsf{listC}(j,\ v \cdot \beta)\}$

$\{I \equiv \exists \alpha, \beta.\ \mathsf{listC}(i,\alpha) * \mathsf{listC}(j,\beta) \wedge \alpha_0 = \mathrm{rev}(\beta) \cdot \alpha\}$

take $\alpha', v \cdot \beta$: $\alpha_0 = \mathrm{rev}(\beta) \cdot v \cdot \alpha' = \mathrm{rev}(v \cdot \beta) \cdot \alpha'$ ✓

Proving VC2: one lap

$\{\mathsf{listC}(i,\alpha) * \mathsf{listC}(j,\beta) \wedge i \neq \mathsf{nil}\}$ — from $I \wedge i \neq \mathsf{nil}$

$\{i \mapsto v, z * \mathsf{listC}(z,\alpha') * \mathsf{listC}(j,\beta)\}$ — unfold, $\alpha = v \cdot \alpha'$

$k := [i.\mathtt{next}];$ — lookup, frame $\mathsf{listC}(z,\alpha') * \mathsf{listC}(j,\beta)$

$\{i \mapsto v, k * \mathsf{listC}(k,\alpha') * \mathsf{listC}(j,\beta)\}$ — using $k = z$

$[i.\mathtt{next}] := j;$ — mutation, frame $\mathsf{listC}(k,\alpha') * \mathsf{listC}(j,\beta)$

$\{i \mapsto v, j * \mathsf{listC}(k,\alpha') * \mathsf{listC}(j,\beta)\}$

$\{\mathsf{listC}(k,\alpha') * \mathsf{listC}(i,\ v \cdot \beta)\}$ — fold: $i \mapsto v, j * \mathsf{listC}(j,\beta)$

$j := i;\quad i := k$ — assignment, twice

$\{\mathsf{listC}(i,\alpha') * \mathsf{listC}(j,\ v \cdot \beta)\}$

$\{I \equiv \exists \alpha, \beta.\ \mathsf{listC}(i,\alpha) * \mathsf{listC}(j,\beta) \wedge \alpha_0 = \mathrm{rev}(\beta) \cdot \alpha\}$

take $\alpha', v \cdot \beta$: $\alpha_0 = \mathrm{rev}(\beta) \cdot v \cdot \alpha' = \mathrm{rev}(v \cdot \beta) \cdot \alpha'$ ✓

Side conditions ✓: the lookup modifies only k; the mutation modifies nothing.

Where the frame rule earned its keep

The mutation $[i.\texttt{next}] := j$ used the small axiom on a single cell:

$$\{i \mapsto v, _\}\ \ [i.\texttt{next}] := j\ \ \{i \mapsto v, j\}$$

Where the frame rule earned its keep

The mutation $[i.\texttt{next}] := j$ used the small axiom on a single cell:

$$\{i \mapsto v, _\}\ \ [i.\texttt{next}] := j\ \ \{i \mapsto v, j\}$$

Everything else in the heap was framed out:

$$\{(i \mapsto v, _) * \underbrace{\mathsf{listC}(k, \alpha') * \mathsf{listC}(j, \beta)}_{\text{framed}}\}\ [i.\texttt{next}] := j\ \{(i \mapsto v, j) * \mathsf{listC}(k, \alpha') * \mathsf{listC}(j, \beta)\}$$

Separation logic at scale: Infer

Infer

A static analyser from Meta, open source since 2015 (`fbinfer.com`).

- Built on separation logic. Began as a startup (Monoidics) by Calcagno, Distefano, O'Hearn; acquired by Facebook in 2013.
- Runs on code changes at Meta.
- Java, C, C++, Objective-C. Finds null dereferences, memory and resource leaks.



The Monoidics team at Facebook. Photo: QMUL Theory group.

Infer on a list program

```
#include <stdlib.h>
struct node { int val; struct node *next; };

int second(struct node *x) {
  return x->next->val;
}

int caller(void) {
  struct node n = {1, NULL};
  return second(&n);            // null pointer dereference
}

struct node *push(struct node *l, int v) {
  struct node *n = malloc(sizeof *n);
  if (n == NULL) return l;
  n->val = v;  n->next = l;
  return n;
}

void drop(void) {
  push(NULL, 1);                // memory leak
}
```

- No annotations: no pre/post, no invariants.
- Infer infers a spec for each function, in separation logic.
- At a call, the callee's spec is reused; the rest of the heap is framed.

Running it

```
$ infer run -- gcc -c list.c
```

Running it

```
$ infer run -- gcc -c list.c



list.c:10: error: Null Dereference
  `n.next` could be null (null value originating from line 9) and is
  dereferenced in the call to `second()`.
  10.   return second(&n);            // null pointer dereference
                ^

list.c:21: error: Memory Leak
  Memory dynamically allocated by `malloc`, indirectly via call to `push()`
  on line 21 is not freed after the last access at line 21, column 3.
  21.   push(NULL, 1);                // memory leak
        ^

Found 2 issues
```

Wrap

Today

- Inductive predicates — $\mathsf{lseg}(x, y)$, $\mathsf{list}(x)$ — describe unbounded heap structures.
  $*$ in the definition bakes in well-formedness.
- Proof rhythm: unfold, operate, fold.
- In-place list reversal, proved in full.
- Infer: separation logic, at Meta scale, with no annotations.

Next time

We have been proving programs correct — with invariants we supply.

Next time: instead of *proving correctness* we try to find bugs.

Next time

We have been proving programs correct — with invariants we supply.

Next time: instead of *proving correctness* we try to find bugs.

Symbolic execution (Lecture 12): the forward dual of everything so far.

Questions?

ATiPL 2026 Minute Sheet